LAPORAN PROYEK AKHIR
NetCrowd

NRP & NAMA	:	3223600050 Labib Al Afaf
	:	3223600058 Ihsanta Zaki Sanjaya
KELAS	:	2 D4 Teknik Komputer B
DOSEN	:	Maretha Ruswiansari S.ST., M.T
	:	A.R. Anom Besari, Ph.D.
MATA KULIAH	:	Jaringan Komputer
TANGGAL	:	Kamis, 5 Juni 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2025

NetCrowd

I. Latar Belakang

Kualitas jaringan internet merupakan komponen krusial dalam mendukung proses belajar mengajar, khususnya dalam lingkungan kampus yang semakin bergantung pada layanan digital. Namun, di kampus PENS, permasalahan lambatnya koneksi internet telah menjadi keluhan umum di kalangan mahasiswa.

Meskipun keluhan dari pengguna, baik mahasiswa maupun dosen, semakin banyak, tidak ada sistem yang dapat secara sistematis merekam, mengolah, dan memvisualisasikan data kualitas jaringan berdasarkan pengalaman nyata para pengguna. Padahal, konsep seperti ini bukanlah hal baru di dunia teknologi. Platform seperti *Ookla Consumer QoE*™ telah berhasil menerapkan pendekatan crowdsourced untuk mengukur kualitas pengalaman pengguna dalam skala nasional hingga global. Namun, belum ada upaya yang secara khusus mengadaptasikan metode serupa dalam lingkup yang lebih kecil seperti kampus, dengan prinsip transparansi dan integrasi ke dalam sistem kampus yang sudah ada.

Oleh karena itu, dibutuhkan sebuah sistem feedback crowdsourced yang mampu menangkap kondisi jaringan berdasarkan laporan pengguna secara langsung. Sistem ini diharapkan dapat memberikan gambaran realtime dan historis mengenai performa jaringan di berbagai titik dalam kampus, serta menjadi bahan pertimbangan bagi pihak pengelola untuk melakukan perbaikan yang lebih tepat sasaran dan datadriven.

II. Tujuan dan manfaat

Tujuan

Pengembangan sistem NetCrowd memiliki tujuan sebagai berikut:

- Menyediakan platform crowdsourced yang memungkinkan mahasiswa dan civitas kampus untuk melaporkan pengalaman kualitas jaringan secara langsung dan terstruktur.
- Mengumpulkan dan mengolah data laporan jaringan dari berbagai titik di area kampus untuk menghasilkan peta kualitas jaringan (QoE) berbasis lokasi secara realtime.
- Mendukung proses pengambilan keputusan oleh pihak pengelola jaringan kampus dengan data yang akurat, terklaster, dan tervisualisasi.
- Meningkatkan transparansi dan akuntabilitas dalam pengelolaan infrastruktur jaringan kampus, khususnya dalam konteks perbaikan dan alokasi sumber daya.
- Mendorong efisiensi perbaikan dan upgrade jaringan melalui sistem prioritas berdasarkan jumlah dan distribusi laporan pengguna.

Manfaat

Implementasi sistem NetCrowd di lingkungan kampus diharapkan dapat memberikan manfaat sebagai berikut:

- Identifikasi cepat titik masalah jaringan segera setelah laporan masuk dari pengguna.
- Prioritas penanganan yang lebih akurat berdasarkan lokasi dengan jumlah laporan terbanyak.
- Penggunaan sumber daya lebih efisien, baik dari sisi teknis maupun anggaran, dengan fokus pada area kritis.

- Meningkatkan transparansi dan keterlibatan pengguna, karena setiap laporan berkontribusi langsung terhadap pemetaan kualitas jaringan.
- Mendukung business case perbaikan atau upgrade jaringan ke manajemen kampus dengan bukti berbasis data visual dan kuantitatif.
- Mengurangi ketergantungan pada data teknis internal saja (misalnya dari perangkat WLC/router), dengan menambahkan perspektif pengalaman pengguna secara langsung.
- Mencegah bias atau spam data melalui validasi, clustering otomatis, dan mekanisme threshold pelaporan.

III. Fiturfitur dalam NetCrowd

Fitur Utama Aplikasi NetCrowd

Aplikasi NetCrowd dirancang dengan fokus pada pelaporan kualitas jaringan berbasis pengalaman pengguna, serta dilengkapi dengan fitur visualisasi dan manajemen data untuk mendukung pengambilan keputusan. Fiturfitur utamanya meliputi:

1. Fitur Pelaporan Kualitas Jaringan (Network Report)

   Pengguna dapat mengirimkan laporan kondisi jaringan secara langsung melalui antarmuka web atau integrasi WhatsApp, dengan parameter seperti lokasi, waktu, dan deskripsi masalah.

2. Laporan Berdasarkan Gedung (Report by Building)

   Sistem mengelompokkan laporan berdasarkan lokasi gedung untuk memudahkan identifikasi area dengan masalah jaringan tertinggi.

3. Laporan Mingguan Otomatis (Weekly Timed Report)

   Aplikasi menyusun rekap laporan secara berkala (mingguan) untuk memantau tren dan pola masalah jaringan secara waktu nyata.

4. Penentuan Prioritas Gedung (Priority Building)

   Sistem secara otomatis memprioritaskan area atau gedung dengan jumlah laporan terbanyak untuk didahulukan dalam penanganan.

5. Distribusi Jenis Masalah (Issue Type Distribution)

   Visualisasi statistik jenisjenis keluhan jaringan yang dilaporkan, seperti koneksi lambat, putusputus, atau tidak terhubung.

6. Peta Kampus Interaktif (Interactive Campus Map)

   Visualisasi laporan secara spasial pada peta 2D kampus yang interaktif, sehingga pengguna dan pengelola dapat melihat titiktitik masalah secara realtime.

7. Informasi Detail Gedung (Building Details)

Setiap titik pada peta dapat diklik untuk menampilkan rincian laporan, statistik, dan riwayat gangguan di gedung tersebut.

8. Manajemen Laporan (Report Management)

   Fitur backend untuk admin atau teknisi jaringan dalam memverifikasi, memfilter, dan menindaklanjuti laporan yang masuk, termasuk validasi dan penandaan status laporan (baru, diproses, selesai).

IV. Panduan

Aplikasi NetCrowd dirancang untuk memudahkan pelaporan dan pengelolaan masalah jaringan di lingkungan kampus secara kolaboratif. Berikut merupakan alur penggunaan aplikasi dari sudut pandang pengguna umum dan admin/pengelola:

1. Pengguna Umum (Mahasiswa/Dosen)

   a. Melakukan Pelaporan
   Pengguna memiliki dua opsi untuk mengirimkan laporan gangguan jaringan:

   - Melalui pengisian formulir di web: Pengguna mengisi lokasi, jenis masalah, dan deskripsi.
   - Melalui WhatsApp Chat: Sistem menerima format laporan yang dikirim via WhatsApp dan secara otomatis mencatatnya.

   b. Konfirmasi Pengiriman
   Setelah laporan dikirim, sistem akan memberikan notifikasi bahwa laporan telah berhasil diterima dan dicatat di sistem.

   c. Melihat Laporan Terkini
   Pengguna dapat melihat laporannya muncul di bagian Recent Network Report, yang menampilkan data laporan terbaru berdasarkan waktu dan lokasi.

2. Admin/Pengelola Jaringan

   a. Login Admin
   Admin atau pihak teknis melakukan login ke dashboard menggunakan akun yang telah diberikan.

   b. Melihat Jumlah Laporan
   Di halaman utama dashboard, admin dapat melihat rekap jumlah laporan per gedung. Misalnya, sistem menunjukkan bahwa Gedung SAW memiliki 3 laporan aktif.

   c. Menentukan Prioritas Penanganan
   Berdasarkan sistem penilaian otomatis, gedung dengan jumlah laporan tertinggi akan masuk dalam daftar prioritas perbaikan. Dalam contoh ini, Gedung SAW menjadi prioritas utama.

d. Melihat Peta Interaktif
Fitur Interactive Campus Map menampilkan titiktitik lokasi yang menerima laporan. Titiktitik tersebut divisualisasikan berdasarkan koordinat lokasi dalam kampus.

e. Manajemen Laporan
Melalui fitur Report Management, admin dapat:

- Melihat detail laporan.
- Mengubah status laporan menjadi *"Sedang Diproses"* atau *"Selesai/Resolved"* setelah tindakan dilakukan.

3. Pembaruan Visual dan Data Sistem
Ketika laporan ditandai sebagai Resolved, maka:

- Warna indikator bar pada sistem berubah menjadi hijau untuk menandakan laporan telah ditangani.
- Skor prioritas pada gedung tersebut akan disesuaikan ulang.
- Status pada Recent Network Report dan Building Details juga ikut diperbarui secara otomatis untuk menjaga konsistensi data.

V. **Program**
Database:
Database Ts

```
import type { Report, LocationStats } from "./types"

// Inmemory database simulation with multiple storage options
class Database {
  private static instance: Database
  private reports: Report[] = []
  private listeners: Set<() => void> = new Set()
  private isInitialized = false
  private storageKey = "netcrowd_reports_v2"

  static getInstance(): Database {
    if (!Database.instance) {
      Database.instance = new Database()
    }
    return Database.instance
  }

  constructor() {
    if (typeof window !== "undefined") {
      this.loadFromStorage()
      this.initializeDefaultData()
    }
  }

  private loadFromStorage() {
    if (typeof window !== "undefined") {
      // Try to load from localStorage first
      let stored = localStorage.getItem(this.storageKey)
```

```
      // If not found, try to load from sessionStorage (for temporary
session)
      if (!stored) {
        stored = sessionStorage.getItem(this.storageKey)
      }

      if (stored) {
        try {
          const data = JSON.parse(stored)
          this.reports = data.map((report: any) => ({
            ...report,
            createdAt: new Date(report.createdAt),
            updatedAt: new Date(report.updatedAt),
          }))
          this.isInitialized = true
                  console.log("Loaded   reports   from   storage:",
this.reports.length)
        } catch (error) {
            console.error("Failed to load reports from storage:",
error)
          this.reports = []
        }
      }
    }
  }

  private saveToStorage() {
    if (typeof window !== "undefined") {
      try {
        const dataString = JSON.stringify(this.reports)

           //  Save  to  both  localStorage  and  sessionStorage  for
redundancy
        localStorage.setItem(this.storageKey, dataString)
        sessionStorage.setItem(this.storageKey, dataString)

        console.log("Data saved to storage:", this.reports.length,
"reports")
      } catch (error) {
        console.error("Failed to save to storage:", error)
      }
    }
  }

  // Method to export data for sharing between browsers/devices
  exportData(): string {
    return JSON.stringify(
      {
        reports: this.reports,
        exportedAt: new Date().toISOString(),
        version: "2.0",
```

```
      },
      null,
      2,
    )
  }

  // Method to import data from another browser/device
  importData(jsonData: string): { success: boolean; message: string;
imported: number } {
    try {
      const data = JSON.parse(jsonData)

      if (!data.reports || !Array.isArray(data.reports)) {
         return { success: false, message: "Invalid data format",
imported: 0 }
      }

      const importedReports = data.reports.map((report: any) => ({
        ...report,
        createdAt: new Date(report.createdAt),
        updatedAt: new Date(report.updatedAt),
      }))

      // Merge with existing data, avoiding duplicates
      const existingIds = new Set(this.reports.map((r) => r.id))
         const  newReports  =  importedReports.filter((r:  Report)
=> !existingIds.has(r.id))

      this.reports = [...this.reports, ...newReports]
      this.saveToStorage()
      this.notifyListeners()

      return {
        success: true,
         message: `Successfully imported ${newReports.length} new
reports`,
        imported: newReports.length,
      }
    } catch (error) {
      return {
        success: false,
       message: "Failed to parse data: " + (error instanceof Error ?
error.message : "Unknown error"),
        imported: 0,
      }
    }
  }

  // Method to sync data across tabs/windows
  syncAcrossTabs() {
    if (typeof window !== "undefined") {
      // Listen for storage changes from other tabs
```

```
      window.addEventListener("storage", (e) => {
        if (e.key === this.storageKey && e.newValue) {
          try {
            const data = JSON.parse(e.newValue)
            this.reports = data.map((report: any) => ({
              ...report,
              createdAt: new Date(report.createdAt),
              updatedAt: new Date(report.updatedAt),
            }))
            this.notifyListeners()
                  console.log("Synced  data  from  another  tab:",
this.reports.length, "reports")
          } catch (error) {
             console.error("Failed to sync data from another tab:",
error)
          }
        }
      })
    }
  }

  private notifyListeners() {
        console.log("Notifying   listeners,   current   reports:",
this.reports.length)
    this.listeners.forEach((listener) => {
      try {
        listener()
      } catch (error) {
        console.error("Error in listener:", error)
      }
    })
  }

  subscribe(listener: () => void) {
    this.listeners.add(listener)
    return () => this.listeners.delete(listener)
  }

  private formatTimestamp(date: Date): string {
    return date.toLocaleString("idID", {
      year: "numeric",
      month: "2digit",
      day: "2digit",
      hour: "2digit",
      minute: "2digit",
    })
  }

  private initializeDefaultData() {
    if (this.reports.length === 0 && !this.isInitialized) {
      const now = new Date()
      const yesterday = new Date(now.getTime()  24 * 60 * 60 * 1000)
```

```
      const defaultReports: Report[] = [
        {
          id: "RPT001",
          name: "Ahmad Rizki",
          email: "ahmad.rizki@student.ac.id",
          building: "SAW",
          room: "Cafeteria",
          issueType: "Slow Internet Speed",
          device: "Laptop",
          description:
            "Internet speed is very slow, cannot load YouTube videos
or download files. Speed test shows only 1 Mbps.",
          severity: "medium",
          status: "investigating",
          timestamp: this.formatTimestamp(new Date(now.getTime()  4
* 60 * 60 * 1000)),
          createdAt: new Date(now.getTime()  4 * 60 * 60 * 1000),
          updatedAt: new Date(now.getTime()  3 * 60 * 60 * 1000),
        },
        {
          id: "RPT002",
          name: "Sarah Putri",
          email: "sarah.putri@student.ac.id",
          building: "SAW",
          room: "Library",
          issueType: "No Internet Connection",
          device: "Smartphone",
             description: "Cannot connect to campus WiFi. Shows
connected but no internet access.",
          severity: "high",
          status: "new",
          timestamp: this.formatTimestamp(new Date(now.getTime()  5
* 60 * 60 * 1000)),
          createdAt: new Date(now.getTime()  5 * 60 * 60 * 1000),
          updatedAt: new Date(now.getTime()  5 * 60 * 60 * 1000),
        },
        {
          id: "RPT003",
          name: "Budi Santoso",
          email: "budi.santoso@student.ac.id",
          building: "D3",
          room: "HI101",
          issueType: "Intermittent Connection",
          device: "Laptop",
         description: "Connection keeps dropping every few minutes.
Very disruptive during online classes.",
          severity: "high",
          status: "resolved",
                              timestamp:    this.formatTimestamp(new
Date(yesterday.getTime()  6 * 60 * 60 * 1000)),
           createdAt: new Date(yesterday.getTime()  6 * 60 * 60 *
```

```
1000),
          updatedAt: yesterday,
        },
        {
          id: "RPT004",
          name: "Maya Sari",
          email: "maya.sari@student.ac.id",
          building: "D4",
          room: "Lab D4A",
          issueType: "High Latency/Ping",
          device: "Desktop Computer",
           description: "High ping when accessing online resources.
Gaming and video calls are affected.",
          severity: "medium",
          status: "resolved",
          timestamp: this.formatTimestamp(new Date(now.getTime()  7
* 60 * 60 * 1000)),
          createdAt: new Date(now.getTime()  7 * 60 * 60 * 1000),
          updatedAt: now,
        },
      ]

      this.reports = defaultReports
      this.saveToStorage()
      this.isInitialized = true
              console.log("Initialized   with   default   data:",
this.reports.length, "reports")
    }

    // Enable crosstab sync
    this.syncAcrossTabs()
  }

  // CRUD Operations (unchanged)
  getAllReports(filters?: {
    building?: string
    status?: string
    severity?: string
  }): Report[] {
    let filteredReports = [...this.reports]

    if (filters?.building && filters.building !== "all") {
          filteredReports  =  filteredReports.filter((report)  =>
report.building === filters.building)
    }

    if (filters?.status && filters.status !== "all") {
          filteredReports  =  filteredReports.filter((report)  =>
report.status === filters.status)
    }

    if (filters?.severity && filters.severity !== "all") {
```

```
          filteredReports  =  filteredReports.filter((report)  =>
report.severity === filters.severity)
    }

           return    filteredReports.sort((a,    b)    =>    new
Date(b.createdAt).getTime()  new Date(a.createdAt).getTime())
  }

  getReportById(id: string): Report | null {
    return this.reports.find((report) => report.id === id) || null
  }

    createReport(reportData:  Omit<Report,  "id"  |  "status"  |
"timestamp" | "createdAt" | "updatedAt">): Report {
    const now = new Date()

    // Generate unique ID
    const existingIds = this.reports.map((r) => r.id)
     let newId = `RPT${String(this.reports.length + 1).padStart(3,
"0")}`
    let counter = this.reports.length + 1

    while (existingIds.includes(newId)) {
      counter++
      newId = `RPT${String(counter).padStart(3, "0")}`
    }

    const newReport: Report = {
      ...reportData,
      id: newId,
      status: "new",
      timestamp: this.formatTimestamp(now),
      createdAt: now,
      updatedAt: now,
    }

    this.reports.push(newReport)
       console.log("Created  new  report:",  newReport.id,  "Total
reports:", this.reports.length)
    this.saveToStorage()
    this.notifyListeners()

    return newReport
  }

  updateReport(id: string, updates: Partial<Report>): Report | null
{
    const reportIndex = this.reports.findIndex((report) => report.id
=== id)

    if (reportIndex === 1) {
      console.error("Report not found:", id)
```

```
      return null
    }

    const now = new Date()
    this.reports[reportIndex] = {
      ...this.reports[reportIndex],
      ...updates,
      updatedAt: now,
    }

        console.log("Updated   report:",   id,   "New   status:",
updates.status)
    this.saveToStorage()
    this.notifyListeners()

    return this.reports[reportIndex]
  }

  deleteReport(id: string): boolean {
    const reportIndex = this.reports.findIndex((report) => report.id
=== id)

    if (reportIndex === 1) {
      console.error("Report not found for deletion:", id)
      return false
    }

    this.reports.splice(reportIndex, 1)
      console.log("Deleted  report:",  id,  "Remaining  reports:",
this.reports.length)
    this.saveToStorage()
    this.notifyListeners()

    return true
  }

  // Analytics methods (unchanged)
  getLocationStats(): LocationStats[] {
    const stats: { [key: string]: LocationStats } = {}

    this.reports.forEach((report) => {
      if (!stats[report.building]) {
        stats[report.building] = {
          building: report.building,
          totalReports: 0,
          resolvedReports: 0,
          activeReports: 0,
          rooms: [],
        }
      }

      stats[report.building].totalReports++
```

```
      if (report.status === "resolved") {
        stats[report.building].resolvedReports++
      } else if (![“false_report”].includes(report.status)) {
        stats[report.building].activeReports++
      }

      const existingRoom = stats[report.building].rooms.find((r) =>
r.room === report.room)
      if (existingRoom) {
        existingRoom.reports++
        if (report.status === "resolved") {
          existingRoom.resolved++
        }
      } else {
        stats[report.building].rooms.push({
          room: report.room,
          reports: 1,
          resolved: report.status === "resolved" ? 1 : 0,
        })
      }
    })

      return  Object.values(stats).sort((a,  b)  =>  b.activeReports
a.activeReports)
  }

  getReportStats() {
    const total = this.reports.length
      const  resolved  =  this.reports.filter((r)  =>  r.status  ===
"resolved").length
      const  active  =  this.reports.filter((r)  =>  ![“resolved”,
"false_report”].includes(r.status)).length

    const today = new Date()
    const todayString = today.toDateString()
    const resolvedToday = this.reports.filter(
              (r)    =>    r.status    ===    "resolved"    &&    new
Date(r.updatedAt).toDateString() === todayString,
    ).length

    return {
      total,
      active,
      resolved,
      resolvedToday,
       resolutionRate: total > 0 ? Math.round((resolved / total) *
100) : 0,
    }
  }

  getWeeklyTrend() {
    const now = new Date()
```

```
    const weekData = []

    for (let i = 6; i >= 0; i) {
      const date = new Date(now.getTime()  i * 24 * 60 * 60 * 1000)
       const dayName = date.toLocaleDateString("idID", { weekday:
"short" })

      const dayReports = this.reports.filter(
         (report) => new Date(report.createdAt).toDateString() ===
date.toDateString(),
      ).length

      const dayResolved = this.reports.filter(
            (report) => report.status === "resolved" && new
Date(report.updatedAt).toDateString() === date.toDateString(),
      ).length

      weekData.push({
        day: dayName,
        reports: dayReports,
        resolved: dayResolved,
      })
    }

    return weekData
  }

  getIssueTypesDistribution() {
    const issueTypes: { [key: string]: number } = {}

    this.reports.forEach((report) => {
       issueTypes[report.issueType] = (issueTypes[report.issueType]
|| 0) + 1
    })

     const colors = ["#3b82f6", "#ef4444", "#f59e0b", "#10b981",
"#8b5cf6", "#f97316"]

    return Object.entries(issueTypes).map(([name, value], index) =>
({
       name: name.replace(" Internet", "").replace(" Connection",
""),
      value,
      color: colors[index % colors.length],
    }))
  }

  clearAllData() {
    this.reports = []
    this.saveToStorage()
    this.notifyListeners()
  }
```

```
  resetToDefault() {
    this.reports = []
    this.isInitialized = false
    this.initializeDefaultData()
    this.notifyListeners()
  }
}

export const database = Database.getInstance()
```

Program ini merupakan simulasi database lokal berbasis TypeScript yang berfungsi untuk mengelola laporan masalah jaringan. Data laporan disimpan secara sementara dalam memori (in-memory) dan juga disimpan secara permanen di localStorage dan sessionStorage browser agar tetap tersedia saat halaman dimuat ulang. Program ini dirancang sebagai singleton class, sehingga hanya ada satu instance Database yang digunakan selama aplikasi berjalan.

Fitur utamanya meliputi pembuatan, pembacaan, pembaruan, dan penghapusan laporan (CRUD), ekspor dan impor data dalam format JSON untuk berbagi antar perangkat, serta sinkronisasi data secara otomatis antar tab browser menggunakan event storage. Selain itu, program ini juga menyediakan analitik seperti statistik laporan berdasarkan lokasi, tren mingguan, dan distribusi jenis masalah. Saat pertama kali dijalankan dan belum ada data, program akan menginisialisasi beberapa data laporan default sebagai contoh.

Untuk mendukung responsivitas antarmuka pengguna, program menyediakan mekanisme subscribe() yang memungkinkan komponen lain menerima notifikasi saat data berubah. Secara keseluruhan, program ini cocok digunakan sebagai backend simulasi dalam pengembangan aplikasi pelaporan gangguan jaringan di lingkungan kampus atau institusi, terutama saat membangun prototipe atau aplikasi berbasis web yang mendukung mode offline.

lib/types.ts

```
export interface Report {
  id: string
  name: string
  email?: string
  building: string
  room: string
  issueType: string
  device: string
  description: string
  severity: "low" | "medium" | "high" | "critical"
   status:  "new"  |  "verified"  |  "investigating"  |  "resolved"  |
"false_report"
  timestamp: string
  createdAt: Date
  updatedAt: Date
}

export interface BuildingRooms {
```

```
  [key: string]: string[]
}

export interface LocationStats {
  building: string
  totalReports: number
  resolvedReports: number
  activeReports: number
  rooms: {
    room: string
    reports: number
    resolved: number
  }[]
}
```

Program ini mendefinisikan tiga buah interface yang digunakan untuk merepresentasikan struktur data dalam aplikasi pelaporan masalah jaringan. Interface-interface ini digunakan sebagai kontrak tipe data agar penulisan dan pengelolaan data menjadi konsisten, aman, dan mudah dikembangkan. Berikut penjelasan dalam bentuk paragraf:

Interface Report mendefinisikan struktur sebuah laporan, termasuk atribut seperti id, name, building, room, issueType, device, description, serta tingkat keparahan (severity) dan status penanganan (status) laporan. Terdapat juga atribut createdAt, updatedAt, dan timestamp yang mencatat waktu pelaporan dan perubahan. Properti email bersifat opsional. Interface ini dipakai dalam seluruh operasi CRUD pada sistem. Sementara itu, BuildingRooms adalah representasi objek dinamis yang memetakan nama gedung ke daftar nama ruangan di dalamnya, digunakan untuk tujuan seperti filter atau dropdown pilihan lokasi. Sedangkan interface LocationStats digunakan untuk menyimpan hasil analisis statistik laporan berdasarkan gedung, seperti total laporan, jumlah laporan yang sudah diselesaikan, jumlah laporan aktif, serta statistik per ruangan di gedung tersebut.

Secara keseluruhan, ketiga interface ini digunakan sebagai pondasi struktur data dalam aplikasi pelaporan, memastikan data konsisten dan memungkinkan analisis serta pengolahan informasi yang lebih mudah dan terstruktur.

Component:
components/reportform.tsx

```
"use client"

import type React from "react"

import { useState } from "react"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle }
from "@/components/ui/card"
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import { Textarea } from "@/components/ui/textarea"
```

```
import { Select, SelectContent, SelectItem, SelectTrigger,
SelectValue } from "@/components/ui/select"
import { useToast } from "@/hooks/usetoast"

export function ReportForm() {
  const [isSubmitting, setIsSubmitting] = useState(false)
  const { toast } = useToast()

  const handleSubmit = async (e: React.FormEvent<HTMLFormElement>)
=> {
    e.preventDefault()
    setIsSubmitting(true)

    // Simulate API call
    await new Promise((resolve) => setTimeout(resolve, 1000))

    toast({
      title: "Report submitted successfully!",
       description: "Thank you for helping improve campus network
quality. We'll investigate this issue.",
    })

    setIsSubmitting(false)
    ;(e.target as HTMLFormElement).reset()
  }

  return (
    <Card>
      <CardHeader>
        <CardTitle>Network Issue Report</CardTitle>
        <CardDescription>
            Please provide detailed information about the network
issue you're experiencing
        </CardDescription>
      </CardHeader>
      <CardContent>
        <form onSubmit={handleSubmit} className="spacey4">
          <div className="grid gridcols2 gap4">
            <div className="spacey2">
              <Label htmlFor="name">Your Name</Label>
              <Input id="name" name="name" required />
            </div>
            <div className="spacey2">
              <Label htmlFor="email">Email (Optional)</Label>
              <Input id="email" name="email" type="email" />
            </div>
          </div>

          <div className="spacey2">
            <Label htmlFor="location">Location</Label>
            <Select name="location" required>
              <SelectTrigger>
```

```
                  <SelectValue placeholder="Select campus location"
/>
              </SelectTrigger>
              <SelectContent>
               <SelectItem value="cafeteria">Cafeteria</SelectItem>
                <SelectItem value="library">Library</SelectItem>
              <SelectItem value="buildinga">Building A</SelectItem>
              <SelectItem value="buildingb">Building B</SelectItem>
              <SelectItem value="buildingc">Building C</SelectItem>
                          <SelectItem  value="labcomputer">Computer
Lab</SelectItem>
                                                          <SelectItem
value="auditorium">Auditorium</SelectItem>
               <SelectItem value="dormitory">Dormitory</SelectItem>
                <SelectItem value="other">Other</SelectItem>
              </SelectContent>
            </Select>
          </div>

          <div className="spacey2">
            <Label htmlFor="issuetype">Issue Type</Label>
            <Select name="issueType" required>
              <SelectTrigger>
                <SelectValue placeholder="Select issue type" />
              </SelectTrigger>
              <SelectContent>
                      <SelectItem  value="slowspeed">Slow  Internet
Speed</SelectItem>
                     <SelectItem  value="noconnection">No  Internet
Connection</SelectItem>
                    <SelectItem  value="intermittent">Intermittent
Connection</SelectItem>
                      <SelectItem  value="wifinotworking">WiFi  Not
Working</SelectItem>
                             <SelectItem   value="highlatency">High
Latency/Ping</SelectItem>
                <SelectItem value="other">Other</SelectItem>
              </SelectContent>
            </Select>
          </div>

          <div className="spacey2">
            <Label htmlFor="device">Device Type</Label>
            <Select name="device" required>
              <SelectTrigger>
                <SelectValue placeholder="Select your device" />
              </SelectTrigger>
              <SelectContent>
                <SelectItem value="laptop">Laptop</SelectItem>
                                                          <SelectItem
value="smartphone">Smartphone</SelectItem>
                <SelectItem value="tablet">Tablet</SelectItem>
```

```
                                <SelectItem   value="desktop">Desktop
Computer</SelectItem>
                <SelectItem value="other">Other</SelectItem>
              </SelectContent>
            </Select>
          </div>

          <div className="spacey2">
            <Label htmlFor="description">Issue Description</Label>
            <Textarea
              id="description"
              name="description"
                placeholder="Please describe the network issue in
detail..."
              required
              rows={4}
            />
          </div>

          <div className="spacey2">
            <Label htmlFor="severity">Severity Level</Label>
            <Select name="severity" required>
              <SelectTrigger>
               <SelectValue placeholder="How severe is this issue?"
/>
              </SelectTrigger>
              <SelectContent>
                              <SelectItem   value="low">Low     Minor
inconvenience</SelectItem>
                      <SelectItem  value="medium">Medium    Affects
productivity</SelectItem>
                          <SelectItem  value="high">High     Cannot
work/study</SelectItem>
                  <SelectItem value="critical">Critical   Complete
outage</SelectItem>
              </SelectContent>
            </Select>
          </div>

                      <Button    type="submit"    className="wfull"
disabled={isSubmitting}>
            {isSubmitting ? "Submitting..." : "Submit Report"}
          </Button>
        </form>
      </CardContent>
    </Card>
  )
}
```

Program TypeScript (React) di atas merupakan sebuah komponen form pelaporan masalah jaringan kampus yang dirancang dengan pendekatan antarmuka pengguna berbasis kartu (Card). Komponen ReportForm menggunakan hook useState untuk mengatur status pengiriman (isSubmitting) dan useToast untuk menampilkan notifikasi umpan balik pengguna setelah pengiriman berhasil. Ketika pengguna mengisi dan mengirimkan form, data tidak langsung dikirim ke server, melainkan disimulasikan melalui setTimeout untuk meniru proses asynchronous, sebelum menampilkan notifikasi keberhasilan dan mereset form.

Form ini memiliki berbagai input yang memandu pengguna untuk memberikan informasi lengkap terkait masalah jaringan yang mereka alami. Informasi yang diminta meliputi nama dan email pengguna, lokasi masalah (misalnya perpustakaan, gedung A, laboratorium komputer, dll), jenis masalah (seperti koneksi lambat atau putus-putus), perangkat yang digunakan, deskripsi masalah secara rinci, serta tingkat keparahan masalah dari “low” hingga “critical”. Komponen UI seperti Input, Select, Textarea, dan Button digunakan dari pustaka komponen kustom yang kemungkinan besar berbasis pada Tailwind CSS atau ShadCN UI.

Dengan desain yang intuitif dan terstruktur, form ini membantu pengguna menyampaikan laporan dengan mudah, sekaligus mempermudah tim teknis untuk mengklasifikasikan dan menindaklanjuti laporan berdasarkan lokasi, jenis masalah, perangkat, dan tingkat keparahannya.

components/interactivecampusmap.tsx

```
"use client"

import type React from "react"

import { useState, useRef } from "react"
import { Card, CardContent, CardDescription, CardHeader, CardTitle }
from "@/components/ui/card"
import { Badge } from "@/components/ui/badge"
import { Button } from "@/components/ui/button"
import { Dialog, DialogContent, DialogDescription, DialogHeader,
DialogTitle } from "@/components/ui/dialog"
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import { Select, SelectContent, SelectItem, SelectTrigger,
SelectValue } from "@/components/ui/select"
import { Tabs, TabsContent, TabsList, TabsTrigger } from
"@/components/ui/tabs"
import { Settings, Plus, Edit, Trash2, MapPin } from "lucide-react"
import type { LocationStats } from "@/lib/types"
import type { CampusConfig, BuildingArea, RoomArea } from
"@/lib/campus-config"
import { defaultCampusConfig } from "@/lib/campus-config"
import { useToast } from "@/hooks/use-toast"

interface InteractiveCampusMapProps {
  locationStats: LocationStats[]
  isAdmin?: boolean
}
```

```
export function InteractiveCampusMap({ locationStats, isAdmin =
false }: InteractiveCampusMapProps) {
        const [campusConfig, setCampusConfig] =
useState<CampusConfig>(defaultCampusConfig)
  const [selectedBuilding, setSelectedBuilding] = useState<string |
null>(null)
   const [selectedRoom, setSelectedRoom] = useState<string |
null>(null)
  const [isConfigMode, setIsConfigMode] = useState(false)
     const [showBuildingDialog, setShowBuildingDialog] =
useState(false)
  const [showRoomDialog, setShowRoomDialog] = useState(false)
      const [editingBuilding, setEditingBuilding] =
useState<BuildingArea | null>(null)
   const [editingRoom, setEditingRoom] = useState<RoomArea |
null>(null)
  const [isDrawing, setIsDrawing] = useState(false)
  const [drawStart, setDrawStart] = useState<{ x: number; y: number }
| null>(null)
  const mapRef = useRef<HTMLDivElement>(null)
  const { toast } = useToast()

   const getBuildingColor = (buildingId: string, activeReports:
number) => {
    if (!isConfigMode) {
      if (activeReports >= 5) return "#ef4444" // red-500
      if (activeReports >= 3) return "#f97316" // orange-500
      if (activeReports >= 1) return "#eab308" // yellow-500
      return "#22c55e" // green-500
    }
     const building = campusConfig.buildings.find((b) => b.id ===
buildingId)
    return building?.color || "#6b7280"
  }

  const getBuildingStatus = (activeReports: number) => {
    if (activeReports >= 5) return "Critical"
    if (activeReports >= 3) return "High"
    if (activeReports >= 1) return "Medium"
    return "Good"
  }

  const getCoordinatesFromEvent = (event: React.MouseEvent) => {
    if (!mapRef.current) return { x: 0, y: 0 }
    const rect = mapRef.current.getBoundingClientRect()
    const scaleX = campusConfig.imageWidth / rect.width
    const scaleY = campusConfig.imageHeight / rect.height
    return {
      x: (event.clientX - rect.left) * scaleX,
      y: (event.clientY - rect.top) * scaleY,
    }
```

```
  }

  const handleMapClick = (event: React.MouseEvent) => {
    if (!isConfigMode) return

    const coords = getCoordinatesFromEvent(event)

    if (isDrawing && drawStart) {
      // Finish drawing
      const width = Math.abs(coords.x - drawStart.x)
      const height = Math.abs(coords.y - drawStart.y)
      const x = Math.min(coords.x, drawStart.x)
      const y = Math.min(coords.y, drawStart.y)

      if (width > 10 && height > 10) {
        // Minimum size check
        setEditingBuilding({
          id: "",
          name: "",
          coordinates: { x, y, width, height },
          rooms: [],
          color: "#3b82f6",
        })
        setShowBuildingDialog(true)
      }

      setIsDrawing(false)
      setDrawStart(null)
    } else {
      // Start drawing
      setIsDrawing(true)
      setDrawStart(coords)
    }
  }

  const handleMouseMove = (event: React.MouseEvent) => {
    if (!isDrawing || !drawStart) return
    // This would update a preview rectangle while drawing
  }

  const saveBuildingConfig = (buildingData: Partial<BuildingArea>)
=> {
    if (!editingBuilding) return

    const newBuilding: BuildingArea = {
      ...editingBuilding,
      ...buildingData,
      id: buildingData.id || `building-${Date.now()}`,
    }

    setCampusConfig((prev) => ({
      ...prev,
```

```
      buildings: editingBuilding.id
        ? prev.buildings.map((b) => (b.id === editingBuilding.id ?
newBuilding : b))
        : [...prev.buildings, newBuilding],
    }))

    setShowBuildingDialog(false)
    setEditingBuilding(null)
    toast({
      title: "Building saved",
       description: `Building ${newBuilding.name} has been saved
successfully.`,
    })
  }

  const saveRoomConfig = (roomData: Partial<RoomArea>) => {
    if (!editingRoom || !selectedBuilding) return

    const newRoom: RoomArea = {
      ...editingRoom,
      ...roomData,
      id: roomData.id || `room-${Date.now()}`,
      buildingId: selectedBuilding,
    }

    setCampusConfig((prev) => ({
      ...prev,
      buildings: prev.buildings.map((building) =>
        building.id === selectedBuilding
          ? {
              ...building,
              rooms: editingRoom.id
              ? building.rooms.map((r) => (r.id === editingRoom.id ?
newRoom : r))
                : [...building.rooms, newRoom],
            }
          : building,
      ),
    }))

    setShowRoomDialog(false)
    setEditingRoom(null)
    toast({
      title: "Room saved",
          description:  `Room  ${newRoom.name}  has  been  saved
successfully.`,
    })
  }

  const deleteBuilding = (buildingId: string) => {
    setCampusConfig((prev) => ({
      ...prev,
```

```
      buildings: prev.buildings.filter((b) => b.id !== buildingId),
    }))
    toast({
      title: "Building deleted",
      description: "Building has been removed from the campus map.",
    })
  }

  const deleteRoom = (roomId: string) => {
    setCampusConfig((prev) => ({
      ...prev,
      buildings: prev.buildings.map((building) => ({
        ...building,
        rooms: building.rooms.filter((r) => r.id !== roomId),
      })),
    }))
    toast({
      title: "Room deleted",
      description: "Room has been removed from the building.",
    })
  }

  const selectedBuildingData = selectedBuilding
    ? locationStats.find((stat) => stat.building.toLowerCase() ===
selectedBuilding.toLowerCase())
    : null

     const    selectedBuildingConfig    =    selectedBuilding    ?
campusConfig.buildings.find((b) => b.id === selectedBuilding) : null

  return (
    <div className="space-y-6">
      {isAdmin && (
        <Card>
          <CardHeader>
            <div className="flex justify-between items-center">
              <div>
                <CardTitle>Campus Map Configuration</CardTitle>
                     <CardDescription>Configure  building  and  room
locations on the campus map</CardDescription>
              </div>
              <div className="flex space-x-2">
                <Button
                  variant={isConfigMode ? "default" : "outline"}
                  onClick={() => setIsConfigMode(!isConfigMode)}
                  className="flex items-center space-x-2"
                >
                  <Settings className="w-4 h-4" />
                   <span>{isConfigMode ? "Exit Config" : "Configure
Map"}</span>
                </Button>
              </div>
```

```
            </div>
          </CardHeader>
          {isConfigMode && (
            <CardContent>
              <div className="bg-blue-50 p-4 rounded-lg mb-4">
                <p className="text-sm text-blue-800">
                    <strong>Configuration Mode:</strong> Click and
drag on the map to define building boundaries. Click on
                  existing buildings to edit or add rooms.
                </p>
              </div>
            </CardContent>
          )}
        </Card>
      )}

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Interactive Campus Map */}
        <Card>
          <CardHeader>
            <CardTitle>Interactive Campus Map</CardTitle>
            <CardDescription>
              {isConfigMode
                ? "Click and drag to define building areas"
                 : "Click on buildings to view network status and
details"}
            </CardDescription>
          </CardHeader>
          <CardContent>
            <div
              ref={mapRef}
               className="relative bg-gray-100 rounded-lg overflow-
hidden cursor-crosshair"
                                          style={{     aspectRatio:
`${campusConfig.imageWidth}/${campusConfig.imageHeight}` }}
              onClick={isConfigMode ? handleMapClick : undefined}
                   onMouseMove={isConfigMode  ?  handleMouseMove  :
undefined}
            >
              {/* Campus aerial image */}
              <img
                src="/images/campus-aerial.png"
                alt="Campus Aerial View"
                className="w-full h-full object-cover"
                draggable={false}
              />

              {/* Building overlays */}
              {campusConfig.buildings.map((building) => {
                        const  stats  =  locationStats.find((s)  =>
s.building.toLowerCase() === building.name.toLowerCase())
                const activeReports = stats?.activeReports || 0
```

```
                return (
                  <div key={building.id}>
                    {/* Building area */}
                    <div
                      className={`absolute border-2 ${
                        isConfigMode
                         ? "border-blue-500 bg-blue-200 bg-opacity-
30"
                            : "border-white bg-black bg-opacity-20
hover:bg-opacity-40 cursor-pointer"
                      } transition-all duration-200`}
                      style={{
                                left:  `${(building.coordinates.x  /
campusConfig.imageWidth) * 100}%`,
                                 top:  `${(building.coordinates.y  /
campusConfig.imageHeight) * 100}%`,
                               width: `${(building.coordinates.width /
campusConfig.imageWidth) * 100}%`,
                              height: `${(building.coordinates.height /
campusConfig.imageHeight) * 100}%`,
                        backgroundColor: isConfigMode
                          ? `${building.color}40`
                                 :  `${getBuildingColor(building.id,
activeReports)}40`,
                      }}
                      onClick={(e) => {
                        e.stopPropagation()
                        if (isConfigMode) {
                          setEditingBuilding(building)
                          setShowBuildingDialog(true)
                        } else {
                          setSelectedBuilding(building.id)
                          setSelectedRoom(null)
                        }
                      }}
                    >
                      {/* Building label */}
                       <div className="absolute inset-0 flex items-
center justify-center">
                        <div className="bg-white bg-opacity-90 px-2
py-1 rounded text-xs font-medium text-gray-800">
                          {building.name}
                                  {!isConfigMode  &&  stats  &&  <div
className="text-xs text-red-600">{activeReports} issues</div>}
                        </div>
                      </div>

                      {/* Room overlays */}
                      {!isConfigMode &&
                        building.rooms.map((room) => (
                          <div
```

```
                            key={room.id}
                            className="absolute border border-gray-
400 bg-white bg-opacity-60 hover:bg-opacity-80 cursor-pointer"
                            style={{
                                   left: `${((room.coordinates.x  -
building.coordinates.x) / building.coordinates.width) * 100}%`,
                                    top: `${((room.coordinates.y  -
building.coordinates.y) / building.coordinates.height) * 100}%`,
                                width: `${(room.coordinates.width /
building.coordinates.width) * 100}%`,
                              height: `${(room.coordinates.height /
building.coordinates.height) * 100}%`,
                            }}
                            onClick={(e) => {
                              e.stopPropagation()
                              setSelectedRoom(room.id)
                              setSelectedBuilding(building.id)
                            }}
                            title={room.name}
                          >
                                      <div  className="text-xs  p-1
truncate">{room.name}</div>
                          </div>
                        ))}

                      {/* Config mode room overlays */}
                      {isConfigMode &&
                        building.rooms.map((room) => (
                          <div
                            key={room.id}
                           className="absolute border border-green-
500 bg-green-200 bg-opacity-50 cursor-pointer"
                            style={{
                                   left: `${((room.coordinates.x  -
building.coordinates.x) / building.coordinates.width) * 100}%`,
                                    top: `${((room.coordinates.y  -
building.coordinates.y) / building.coordinates.height) * 100}%`,
                                width: `${(room.coordinates.width /
building.coordinates.width) * 100}%`,
                              height: `${(room.coordinates.height /
building.coordinates.height) * 100}%`,
                            }}
                            onClick={(e) => {
                              e.stopPropagation()
                              setEditingRoom(room)
                              setSelectedBuilding(building.id)
                              setShowRoomDialog(true)
                            }}
                          >
                                      <div  className="text-xs  p-1
truncate">{room.name}</div>
                          </div>
```

```
                        ))}

                      {/* Add room button in config mode */}
                            {isConfigMode  &&  selectedBuilding  ===
building.id && (
                        <Button
                          size="sm"
                          className="absolute bottom-2 right-2"
                          onClick={(e) => {
                            e.stopPropagation()
                            setEditingRoom({
                              id: "",
                              name: "",
                              buildingId: building.id,
                              coordinates: {
                                x: building.coordinates.x + 10,
                                y: building.coordinates.y + 10,
                                width: 30,
                                height: 20,
                              },
                            })
                            setShowRoomDialog(true)
                          }}
                        >
                          <Plus className="w-3 h-3" />
                        </Button>
                      )}
                    </div>

                    {/* Config mode controls */}
                    {isConfigMode && (
                      <div
                        className="absolute flex space-x-1"
                        style={{
                                left:  `${(building.coordinates.x  /
campusConfig.imageWidth) * 100}%`,
                           top: `${((building.coordinates.y - 30) /
campusConfig.imageHeight) * 100}%`,
                        }}
                      >
                        <Button
                          size="sm"
                          variant="outline"
                          onClick={(e) => {
                            e.stopPropagation()
                            setEditingBuilding(building)
                            setShowBuildingDialog(true)
                          }}
                        >
                          <Edit className="w-3 h-3" />
                        </Button>
                        <Button
```

```
                          size="sm"
                          variant="destructive"
                          onClick={(e) => {
                            e.stopPropagation()
                            deleteBuilding(building.id)
                          }}
                        >
                          <Trash2 className="w-3 h-3" />
                        </Button>
                      </div>
                    )}
                  </div>
                )
              })}

              {/* Drawing preview */}
              {isDrawing && drawStart && (
                <div
                  className="absolute border-2 border-dashed border-
blue-500 bg-blue-200 bg-opacity-30 pointer-events-none"
                  style={{
                    left: `${(drawStart.x / campusConfig.imageWidth)
* 100}%`,
                    top: `${(drawStart.y / campusConfig.imageHeight)
* 100}%`,
                     width: "100px",
                     height: "60px",
                  }}
                />
              )}
            </div>

            {/* Legend */}
            {!isConfigMode && (
              <div className="mt-4 flex flex-wrap gap-2">
                <div className="flex items-center space-x-2">
                <div className="w-4 h-4 bg-green-500 rounded"></div>
                  <span className="text-xs">Good (0 issues)</span>
                </div>
                <div className="flex items-center space-x-2">
                          <div  className="w-4  h-4  bg-yellow-500
rounded"></div>
                           <span  className="text-xs">Medium  (1-2
issues)</span>
                </div>
                <div className="flex items-center space-x-2">
                          <div  className="w-4  h-4  bg-orange-500
rounded"></div>
                 <span class="text-xs">High (3-4 issues)</span>
                </div>
                <div class="flex items-center space-x-2">
                 <div class="w-4 h-4 bg-red-500 rounded"></div>
```

```
                            <span  className="text-xs">Critical  (5+
issues)</span>
                </div>
              </div>
            )}
          </CardContent>
        </Card>

        {/* Building/Room Details */}
        <Card>
          <CardHeader>
            <CardTitle>
                      {isConfigMode  ?  "Configuration  Details"  :
selectedBuilding ? "Building Details" : "Campus Overview"}
            </CardTitle>
            <CardDescription>
              {isConfigMode
                ? "Manage building and room configurations"
                : selectedBuilding
                                  ?  `Detailed  information  for
${selectedBuildingConfig?.name || selectedBuilding}`
                  : "Click on a building to view details"}
            </CardDescription>
          </CardHeader>
          <CardContent>
            {isConfigMode ? (
              <Tabs defaultValue="buildings">
                <TabsList>
                                                          <TabsTrigger
value="buildings">Buildings</TabsTrigger>
                  <TabsTrigger value="rooms">Rooms</TabsTrigger>
                </TabsList>
                 <TabsContent value="buildings" className="space-y-
4">
                  <div className="space-y-2">
                    {campusConfig.buildings.map((building) => (
                     <div key={building.id} className="flex justify-
between items-center p-2 bg-gray-50 rounded">
                        <span>{building.name}</span>
                        <div className="flex space-x-2">
                          <Button
                            size="sm"
                            variant="outline"
                            onClick={() => {
                              setEditingBuilding(building)
                              setShowBuildingDialog(true)
                            }}
                          >
                            <Edit className="w-3 h-3" />
                          </Button>
                           <Button size="sm" variant="destructive"
onClick={() => deleteBuilding(building.id)}>
```

```
                            <Trash2 className="w-3 h-3" />
                          </Button>
                        </div>
                      </div>
                    ))}
                  </div>
                </TabsContent>
                <TabsContent value="rooms" className="space-y-4">
                  {selectedBuildingConfig ? (
                    <div className="space-y-2">
                                                <h4   className="font-
medium">{selectedBuildingConfig.name} Rooms</h4>
                      {selectedBuildingConfig.rooms.map((room) => (
                        <div key={room.id} className="flex justify-
between items-center p-2 bg-gray-50 rounded">
                          <span>{room.name}</span>
                          <div className="flex space-x-2">
                            <Button
                              size="sm"
                              variant="outline"
                              onClick={() => {
                                setEditingRoom(room)
                                setShowRoomDialog(true)
                              }}
                            >
                              <Edit className="w-3 h-3" />
                            </Button>
                            <Button size="sm" variant="destructive"
onClick={() => deleteRoom(room.id)}>
                              <Trash2 className="w-3 h-3" />
                            </Button>
                          </div>
                        </div>
                      ))}
                      <Button
                        className="w-full"
                        onClick={() => {
                          setEditingRoom({
                            id: "",
                            name: "",
                            buildingId: selectedBuildingConfig.id,
                            coordinates: {
                            x: selectedBuildingConfig.coordinates.x
+ 10,
                            y: selectedBuildingConfig.coordinates.y
+ 10,
                              width: 30,
                              height: 20,
                            },
                          })
                          setShowRoomDialog(true)
                        }}
```

```
                      >
                        <Plus className="w-4 h-4 mr-2" />
                        Add Room
                      </Button>
                    </div>
                  ) : (
                     <p className="text-gray-500">Select a building
to manage its rooms</p>
                  )}
                </TabsContent>
              </Tabs>
            ) : selectedBuildingData && selectedBuildingConfig ? (
              <div className="space-y-4">
                <div className="flex justify-between items-center">
                                    <h3   className="text-lg   font-
semibold">{selectedBuildingConfig.name}</h3>
                  <Badge
                    variant={
                      selectedBuildingData.activeReports >= 5
                        ? "destructive"
                        : selectedBuildingData.activeReports >= 3
                          ? "default"
                          : selectedBuildingData.activeReports >= 1
                            ? "secondary"
                            : "outline"
                    }
                  >
                     {getBuildingStatus(selectedBuildingData.active
Reports)}
                  </Badge>
                </div>

               <div className="grid grid-cols-3 gap-4 text-center">
                  <div className="bg-blue-50 p-3 rounded-lg">
                      <div className="text-2xl font-bold text-blue-
600">{selectedBuildingData.totalReports}</div>
                       <div  className="text-sm  text-gray-600">Total
Reports</div>
                  </div>
                  <div className="bg-red-50 p-3 rounded-lg">
                       <div  className="text-2xl  font-bold  text-red-
600">{selectedBuildingData.activeReports}</div>
                      <div  className="text-sm  text-gray-600">Active
Issues</div>
                  </div>
                  <div className="bg-green-50 p-3 rounded-lg">
                     <div  className="text-2xl  font-bold  text-green-
600">{selectedBuildingData.resolvedReports}</div>
                                <div  className="text-sm  text-gray-
600">Resolved</div>
                  </div>
                </div>
```

```
                <div>
                        <h4  className="font-medium  mb-2">Room-wise
Issues</h4>
                    <div  className="space-y-2  max-h-48  overflow-y-
auto">
                    {selectedBuildingData.rooms
                         .sort((a,  b)  =>  b.reports  -  b.resolved  -
(a.reports - a.resolved))
                      .map((room) => (
                        <div
                          key={room.room}
                            className={`flex justify-between items-
center p-2 rounded cursor-pointer ${
                               selectedRoom === room.room ? "bg-blue-
100" : "bg-gray-50 hover:bg-gray-100"
                          }`}
                        onClick={() => setSelectedRoom(selectedRoom
=== room.room ? null : room.room)}
                        >
                         <span className="text-sm flex items-center
space-x-2">
                           <MapPin className="w-3 h-3" />
                           <span>{room.room}</span>
                          </span>
                          <div className="flex space-x-2">
                          <Badge variant="outline" className="text-
xs">
                              {room.reports - room.resolved} active
                           </Badge>
                                       <Badge  variant="secondary"
className="text-xs">
                              {room.reports} total
                           </Badge>
                          </div>
                        </div>
                      ))}
                  </div>
                </div>
              </div>
            ) : (
              <div className="text-center text-gray-500 py-8">
               <MapPin className="w-12 h-12 mx-auto mb-4 text-gray-
300" />
                <p>Click on a building in the map to view detailed
information</p>
              </div>
            )}
          </CardContent>
        </Card>
      </div>
```

```
      {/* Building Configuration Dialog */}
                          <Dialog        open={showBuildingDialog}
onOpenChange={setShowBuildingDialog}>
        <DialogContent>
          <DialogHeader>
             <DialogTitle>{editingBuilding?.id ? "Edit Building" :
"Add New Building"}</DialogTitle>
                <DialogDescription>Configure  building  details  and
location on the campus map</DialogDescription>
          </DialogHeader>
          <div className="space-y-4">
            <div>
              <Label htmlFor="building-name">Building Name</Label>
              <Input
                id="building-name"
                defaultValue={editingBuilding?.name}
                placeholder="e.g., D4, D3, TC"
               onChange={(e) => setEditingBuilding((prev) => (prev ?
{ ...prev, name: e.target.value } : null))}
              />
            </div>
            <div>
              <Label htmlFor="building-id">Building ID</Label>
              <Input
                id="building-id"
                defaultValue={editingBuilding?.id}
                placeholder="e.g., d4, d3, tc"
                onChange={(e) =>
                   setEditingBuilding((prev) => (prev ? { ...prev,
id: e.target.value.toLowerCase() } : null))
                }
              />
            </div>
            <div>
             <Label htmlFor="building-color">Building Color</Label>
              <Input
                id="building-color"
                type="color"
                defaultValue={editingBuilding?.color}
               onChange={(e) => setEditingBuilding((prev) => (prev ?
{ ...prev, color: e.target.value } : null))}
              />
            </div>
            <div className="grid grid-cols-4 gap-2">
              <div>
                <Label htmlFor="building-x">X Position</Label>
                <Input
                  id="building-x"
                  type="number"
                  defaultValue={editingBuilding?.coordinates.x}
                  onChange={(e) =>
                    setEditingBuilding((prev) =>
```

```
                      prev
                        ? {
                            ...prev,
                            coordinates: { ...prev.coordinates, x:
Number.parseInt(e.target.value) || 0 },
                          }
                        : null,
                    )
                  }
                />
              </div>
              <div>
                <Label htmlFor="building-y">Y Position</Label>
                <Input
                  id="building-y"
                  type="number"
                  defaultValue={editingBuilding?.coordinates.y}
                  onChange={(e) =>
                    setEditingBuilding((prev) =>
                      prev
                        ? {
                            ...prev,
                            coordinates: { ...prev.coordinates, y:
Number.parseInt(e.target.value) || 0 },
                          }
                        : null,
                    )
                  }
                />
              </div>
              <div>
                <Label htmlFor="building-width">Width</Label>
                <Input
                  id="building-width"
                  type="number"
                  defaultValue={editingBuilding?.coordinates.width}
                  onChange={(e) =>
                    setEditingBuilding((prev) =>
                      prev
                        ? {
                            ...prev,
                               coordinates: { ...prev.coordinates,
width: Number.parseInt(e.target.value) || 0 },
                          }
                        : null,
                    )
                  }
                />
              </div>
              <div>
                <Label htmlFor="building-height">Height</Label>
                <Input
```

```
                  id="building-height"
                  type="number"
                 defaultValue={editingBuilding?.coordinates.height}
                  onChange={(e) =>
                    setEditingBuilding((prev) =>
                      prev
                        ? {
                            ...prev,
                               coordinates: { ...prev.coordinates,
height: Number.parseInt(e.target.value) || 0 },
                          }
                        : null,
                    )
                  }
                />
              </div>
            </div>
            <div className="flex justify-end space-x-2">
                       <Button  variant="outline"  onClick={()  =>
setShowBuildingDialog(false)}>
                Cancel
              </Button>
                                      <Button    onClick={()    =>
saveBuildingConfig(editingBuilding || {})}>Save Building</Button>
            </div>
          </div>
        </DialogContent>
      </Dialog>

      {/* Room Configuration Dialog */}
                               <Dialog         open={showRoomDialog}
onOpenChange={setShowRoomDialog}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>{editingRoom?.id ? "Edit Room" : "Add New
Room"}</DialogTitle>
            <DialogDescription>Configure room details and location
within the building</DialogDescription>
          </DialogHeader>
          <div className="space-y-4">
            <div>
              <Label htmlFor="room-name">Room Name</Label>
              <Input
                id="room-name"
                defaultValue={editingRoom?.name}
                placeholder="e.g., HI-101, Lab A, Cafeteria"
                 onChange={(e) => setEditingRoom((prev) => (prev ?
{ ...prev, name: e.target.value } : null))}
              />
            </div>
            <div>
              <Label htmlFor="room-id">Room ID</Label>
```

```
                <Input
                  id="room-id"
                  defaultValue={editingRoom?.id}
                  placeholder="e.g., hi-101, lab-a, cafeteria"
                  onChange={(e) =>
                     setEditingRoom((prev) => (prev ? { ...prev, id:
e.target.value.toLowerCase() } : null))
                  }
                />
              </div>
              <div>
                <Label htmlFor="room-floor">Floor</Label>
                <Select
                  defaultValue={editingRoom?.floor?.toString()}
                  onValueChange={(value) =>
                   setEditingRoom((prev) => (prev ? { ...prev, floor:
Number.parseInt(value) } : null))
                  }
                >
                  <SelectTrigger>
                    <SelectValue placeholder="Select floor" />
                  </SelectTrigger>
                  <SelectContent>
                    <SelectItem value="0">Ground Floor</SelectItem>
                    <SelectItem value="1">1st Floor</SelectItem>
                    <SelectItem value="2">2nd Floor</SelectItem>
                    <SelectItem value="3">3rd Floor</SelectItem>
                  </SelectContent>
                </Select>
              </div>
              <div className="grid grid-cols-4 gap-2">
                <div>
                  <Label htmlFor="room-x">X Position</Label>
                  <Input
                    id="room-x"
                    type="number"
                    defaultValue={editingRoom?.coordinates.x}
                    onChange={(e) =>
                      setEditingRoom((prev) =>
                        prev
                          ? {
                              ...prev,
                              coordinates: { ...prev.coordinates, x:
Number.parseInt(e.target.value) || 0 },
                            }
                          : null,
                      )
                    }
                  />
                </div>
                <div>
                  <Label htmlFor="room-y">Y Position</Label>
```

```
                  <Input
                    id="room-y"
                    type="number"
                    defaultValue={editingRoom?.coordinates.y}
                    onChange={(e) =>
                      setEditingRoom((prev) =>
                        prev
                          ? {
                              ...prev,
                               coordinates: { ...prev.coordinates, y:
Number.parseInt(e.target.value) || 0 },
                            }
                          : null,
                      )
                    }
                  />
                </div>
                <div>
                  <Label htmlFor="room-width">Width</Label>
                  <Input
                    id="room-width"
                    type="number"
                    defaultValue={editingRoom?.coordinates.width}
                    onChange={(e) =>
                      setEditingRoom((prev) =>
                        prev
                          ? {
                              ...prev,
                                 coordinates: { ...prev.coordinates,
width: Number.parseInt(e.target.value) || 0 },
                            }
                          : null,
                      )
                    }
                  />
                </div>
                <div>
                  <Label htmlFor="room-height">Height</Label>
                  <Input
                    id="room-height"
                    type="number"
                    defaultValue={editingRoom?.coordinates.height}
                    onChange={(e) =>
                      setEditingRoom((prev) =>
                        prev
                          ? {
                              ...prev,
                                 coordinates: { ...prev.coordinates,
height: Number.parseInt(e.target.value) || 0 },
                            }
                          : null,
                      )
```

```
                  }
                />
              </div>
            </div>
            <div className="flex justify-end space-x-2">
                      <Button  variant="outline"  onClick={()  =>
setShowRoomDialog(false)}>
                Cancel
              </Button>
              <Button onClick={() => saveRoomConfig(editingRoom ||
{})}>Save Room</Button>
            </div>
          </div>
        </DialogContent>
      </Dialog>
    </div>
  )
}
```

Program di atas adalah sebuah komponen React bernama InteractiveCampusMap yang berfungsi untuk menciptakan dasbor peta kampus yang dinamis dan interaktif. Komponen ini memiliki dua fungsi utama: sebagai alat visualisasi untuk memantau status jaringan di berbagai lokasi dan sebagai alat konfigurasi khusus untuk admin. Dalam mode tampilan standar, peta akan menampilkan gedung-gedung dengan lapisan warna yang merepresentasikan status jaringan saat ini (misalnya, hijau untuk baik, merah untuk kritis) berdasarkan data laporan masalah. Pengguna dapat mengklik sebuah gedung untuk melihat statistik terperinci seperti jumlah laporan aktif dan yang sudah diselesaikan, serta rincian masalah per ruangan. Untuk pengguna dengan hak akses admin, tersedia mode konfigurasi yang memungkinkan mereka untuk secara visual menambahkan, mengedit, atau menghapus area gedung dan ruangan langsung di atas peta menggunakan interaksi *click-and-drag*. Dibuat dengan React dan TypeScript serta memanfaatkan komponen UI dari shadcn/ui, program ini menyediakan antarmuka yang modern dan fungsional untuk manajemen dan pemantauan infrastruktur kampus secara visual.

app/admin/page.tsx

```
"use client"

import { useState, useEffect } from "react"
import { Card, CardContent, CardDescription, CardHeader, CardTitle }
from "@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Badge } from "@/components/ui/badge"
import { Input } from "@/components/ui/input"
import  {  Select,  SelectContent,  SelectItem,  SelectTrigger,
SelectValue } from "@/components/ui/select"
import  {  Tabs,  TabsContent,  TabsList,  TabsTrigger  }  from
"@/components/ui/tabs"
```

```
import { Table, TableBody, TableCell, TableHead, TableHeader,
TableRow } from "@/components/ui/table"
import { Users, AlertTriangle, CheckCircle, Clock, Search, Filter,
RefreshCw, Trash2, AlertCircle } from "lucidereact"
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@/components/ui/alertdialog"
import Link from "next/link"
import { EnhancedCharts } from "@/components/enhancedcharts"
import { InteractiveCampusMap } from
"@/components/interactivecampusmap"
import { DataManagement } from "@/components/datamanagement"
import { ProtectedRoute } from "@/components/auth/protectedroute"
import { UserMenu } from "@/components/auth/usermenu"
import { AuthService } from "@/lib/auth"
import { useToast } from "@/hooks/usetoast"
import { useReports, useLocationStats, useReportStats } from
"@/hooks/usedatabase"

function AdminDashboardContent() {
  const [user, setUser] = useState(null)
  const [activeTab, setActiveTab] = useState("overview")
  const [filters, setFilters] = useState({
    search: "",
    building: "all",
    status: "all",
    severity: "all",
  })
  const { toast } = useToast()

  const authService = AuthService.getInstance()

  // Use custom hooks for realtime data
    const { reports, loading: reportsLoading, refetch:
refetchReports } = useReports(filters)
    const { locationStats, loading: statsLoading, refetch:
refetchStats } = useLocationStats()
  const { stats } = useReportStats()

  useEffect(() => {
    const currentUser = authService.getCurrentUser()
    setUser(currentUser)

    // Get active tab from URL hash
```

```
    const hash = window.location.hash.replace("#", "")
          if    (hash    &&    ["overview",    "map",    "reports",
"data"].includes(hash)) {
      setActiveTab(hash)
    }
  }, [])

  // Update URL when tab changes
  const handleTabChange = (value: string) => {
    setActiveTab(value)
    window.history.replaceState(null, "", `#${value}`)
  }

   const updateReportStatus = async (reportId: string, newStatus:
string) => {
    try {
      const response = await fetch(`/api/reports/${reportId}`, {
        method: "PATCH",
        headers: {
          "ContentType": "application/json",
        },
        body: JSON.stringify({ status: newStatus }),
      })

      if (response.ok) {
        toast({
          title: "Status updated successfully",
              description: `Report ${reportId} status changed to
${newStatus}`,
        })
      } else {
        throw new Error("Failed to update status")
      }
    } catch (error) {
      toast({
        title: "Error updating status",
        description: "Please try again later.",
        variant: "destructive",
      })
    }
  }

  const deleteReport = async (reportId: string) => {
    try {
      const response = await fetch(`/api/reports/${reportId}`, {
        method: "DELETE",
      })

      if (response.ok) {
        toast({
          title: "Report deleted successfully",
            description: `Report ${reportId} has been permanently
```

```
deleted.`,
        })
      } else {
        throw new Error("Failed to delete report")
      }
    } catch (error) {
      toast({
        title: "Error deleting report",
        description: "Please try again later.",
        variant: "destructive",
      })
    }
  }

  const handleLogout = () => {
    window.location.reload()
  }

  const handleRefresh = () => {
    refetchReports()
    refetchStats()
    toast({
      title: "Data refreshed",
        description:  "All  data  has  been  updated  to  the  latest
version.",
    })
  }

  const getStatusColor = (status: string) => {
    switch (status) {
      case "new":
        return "bgblue100 textblue800"
      case "verified":
        return "bgpurple100 textpurple800"
      case "investigating":
        return "bgyellow100 textyellow800"
      case "resolved":
        return "bggreen100 textgreen800"
      case "false_report":
        return "bggray100 textgray800"
      default:
        return "bggray100 textgray800"
    }
  }

  const getSeverityColor = (severity: string) => {
    switch (severity) {
      case "low":
        return "bgblue100 textblue800"
      case "medium":
        return "bgyellow100 textyellow800"
      case "high":
```

```
        return "bgorange100 textorange800"
      case "critical":
        return "bgred100 textred800"
      default:
        return "bggray100 textgray800"
    }
  }

  const filteredReports = reports.filter((report) => {
    const matchesSearch =
      filters.search === "" ||
       report.name.toLowerCase().includes(filters.search.toLowerCas
e()) ||
       report.building.toLowerCase().includes(filters.search.toLowe
rCase()) ||
       report.room.toLowerCase().includes(filters.search.toLowerCas
e()) ||
       report.description.toLowerCase().includes(filters.search.toL
owerCase())

    return matchesSearch
  })

  if (reportsLoading || statsLoading) {
    return (
          <div  className="minhscreen  bggray50  flex  itemscenter
justifycenter">
        <div className="textcenter">
           <div className="animatespin roundedfull h12 w12 borderb2
borderblue600 mxauto"></div>
          <p className="mt4 textgray600">Loading dashboard...</p>
        </div>
      </div>
    )
  }

  return (
    <div className="minhscreen bggray50">
      {/* Header */}
      <header className="bgwhite shadowsm borderb">
        <div className="maxw7xl mxauto px4 sm:px6 lg:px8">
          <div className="flex justifybetween itemscenter py4">
            <div className="flex itemscenter spacex4">
              <Link href="/" className="flex itemscenter spacex2">
                  <img src="/images/logonetcrowd.png" alt="NetCrowd
Logo" className="w8 h8" />
                                <h1   className="textxl   fontbold
textgray900">NetCrowd</h1>
              </Link>
              <span className="textgray400">|</span>
               <h2 className="textlg fontmedium textgray700">Admin
Dashboard</h2>
```

```
            </div>
            <nav className="flex itemscenter spacex4">
                            <Button   variant="outline"   size="sm"
onClick={handleRefresh}>
                <RefreshCw className="w4 h4 mr2" />
                Refresh
              </Button>
              <Link href="/reports">
                <Button variant="outline" size="sm">
                  Public Reports
                </Button>
              </Link>
              <Link href="/">
                <Button variant="outline" size="sm">
                  Back to Home
                </Button>
              </Link>
             {user && <UserMenu user={user} onLogout={handleLogout}
/>}
            </nav>
          </div>
        </div>
      </header>

      <main className="maxw7xl mxauto px4 sm:px6 lg:px8 py8">
        {/* Welcome Message */}
        <div className="mb6">
         <h3 className="text2xl fontbold textgray900">Welcome back,
{user?.username}!</h3>
            <p  className="textgray600">Manage  network  reports  and
monitor campus connectivity status.</p>
        </div>

        {/* Stats Overview */}
        <div className="grid gridcols1 md:gridcols4 gap6 mb8">
          <Card>
                 <CardHeader  className="flex  flexrow  itemscenter
justifybetween spacey0 pb2">
                   <CardTitle  className="textsm  fontmedium">Total
Reports</CardTitle>
              <Users className="h4 w4 textmutedforeground" />
            </CardHeader>
            <CardContent>
              <div className="text2xl fontbold">{stats.total}</div>
                <p  className="textxs  textmutedforeground">All  time
reports</p>
            </CardContent>
          </Card>

          <Card>
                 <CardHeader  className="flex  flexrow  itemscenter
justifybetween spacey0 pb2">
```

```
                    <CardTitle  className="textsm  fontmedium">Active
Issues</CardTitle>
               <AlertTriangle className="h4 w4 textmutedforeground"
/>
            </CardHeader>
            <CardContent>
             <div className="text2xl fontbold">{stats.active}</div>
                  <p  className="textxs  textmutedforeground">Needs
attention</p>
            </CardContent>
          </Card>

          <Card>
                  <CardHeader  className="flex  flexrow  itemscenter
justifybetween spacey0 pb2">
                  <CardTitle  className="textsm  fontmedium">Resolved
Today</CardTitle>
              <CheckCircle className="h4 w4 textmutedforeground" />
            </CardHeader>
            <CardContent>
                                          <div     className="text2xl
fontbold">{stats.resolvedToday}</div>
             <p className="textxs textmutedforeground">Issues fixed
today</p>
            </CardContent>
          </Card>

          <Card>
                  <CardHeader  className="flex  flexrow  itemscenter
justifybetween spacey0 pb2">
                <CardTitle className="textsm fontmedium">Resolution
Rate</CardTitle>
              <Clock className="h4 w4 textmutedforeground" />
            </CardHeader>
            <CardContent>
                                          <div     className="text2xl
fontbold">{stats.resolutionRate}%</div>
                  <p  className="textxs  textmutedforeground">Overall
success rate</p>
            </CardContent>
          </Card>
        </div>

           <Tabs  value={activeTab}  onValueChange={handleTabChange}
className="spacey6">
          <TabsList>
            <TabsTrigger value="overview">Overview</TabsTrigger>
            <TabsTrigger value="map">Campus Map</TabsTrigger>
                              <TabsTrigger    value="reports">Manage
Reports</TabsTrigger>
            <TabsTrigger value="data">Data Management</TabsTrigger>
          </TabsList>
```

```
          <TabsContent value="overview" className="spacey6">
            <EnhancedCharts locationStats={locationStats} />

            {/* Priority Buildings */}
            <Card>
              <CardHeader>
                <CardTitle>Priority Buildings</CardTitle>
                 <CardDescription>Buildings with the highest number
of active issues</CardDescription>
              </CardHeader>
              <CardContent>
                <div className="spacey4">
                  {locationStats
                   .sort((a, b) => b.activeReports  a.activeReports)
                    .slice(0, 5)
                    .map((location, index) => (
                      <div
                        key={location.building}
                         className="flex itemscenter justifybetween
p4 bggray50 roundedlg"
                      >
                        <div className="flex itemscenter spacex3">
                          <div
                                className={`w8 h8 roundedfull flex
itemscenter justifycenter textwhite fontbold ${
                              index === 0 ? "bgred500" : index ===
1 ? "bgorange500" : "bgyellow500"
                            }`}
                          >
                            {index + 1}
                          </div>
                          <div>
                                                                    <p
className="fontmedium">{location.building}</p>
                            <p className="textsm textgray600">
                                {location.activeReports} active of
{location.totalReports} total reports
                            </p>
                          </div>
                        </div>
                        <Badge
                          variant={
                            location.activeReports > 5
                              ? "destructive"
                              : location.activeReports > 2
                                ? "default"
                                : "secondary"
                          }
                        >
                         {location.activeReports > 5 ? "Critical" :
location.activeReports > 2 ? "High" : "Medium"}
```

```
                        </Badge>
                      </div>
                    ))}
                </div>
              </CardContent>
            </Card>
          </TabsContent>

          <TabsContent value="map" className="spacey6">
               <InteractiveCampusMap  locationStats={locationStats}
isAdmin={true} />
          </TabsContent>

          <TabsContent value="reports" className="spacey6">
            {/* Filters */}
            <Card>
              <CardHeader>
                <CardTitle className="flex itemscenter spacex2">
                  <Filter className="w5 h5" />
                  <span>Filter Reports</span>
                </CardTitle>
              </CardHeader>
              <CardContent>
                <div className="grid gridcols1 md:gridcols5 gap4">
                  <div className="relative">
                      <Search className="absolute left3 top3 h4 w4
textgray400" />
                    <Input
                      placeholder="Search reports..."
                      className="pl10"
                      value={filters.search}
                            onChange={(e)  =>  setFilters((prev)  =>
({ ...prev, search: e.target.value }))}
                    />
                  </div>
                  <Select
                    value={filters.building}
                     onValueChange={(value) => setFilters((prev) =>
({ ...prev, building: value }))}
                  >
                    <SelectTrigger>
                      <SelectValue placeholder="Building" />
                    </SelectTrigger>
                    <SelectContent>
                                        <SelectItem   value="all">All
Buildings</SelectItem>
                      <SelectItem value="D4">D4</SelectItem>
                      <SelectItem value="D3">D3</SelectItem>
                      <SelectItem value="TC">TC</SelectItem>
                      <SelectItem value="SAW">SAW</SelectItem>
                                                        <SelectItem
value="Pascasarjana">Pascasarjana</SelectItem>
```

```
                      <SelectItem value="Other">Other</SelectItem>
                    </SelectContent>
                  </Select>
                  <Select
                    value={filters.status}
                     onValueChange={(value) => setFilters((prev) =>
({ ...prev, status: value }))}
                  >
                    <SelectTrigger>
                      <SelectValue placeholder="Status" />
                    </SelectTrigger>
                    <SelectContent>
                    <SelectItem value="all">All Status</SelectItem>
                      <SelectItem value="new">New</SelectItem>
                                                              <SelectItem
value="verified">Verified</SelectItem>
                                                              <SelectItem
value="investigating">Investigating</SelectItem>
                                                              <SelectItem
value="resolved">Resolved</SelectItem>
                            <SelectItem  value="false_report">False
Report</SelectItem>
                    </SelectContent>
                  </Select>
                  <Select
                    value={filters.severity}
                     onValueChange={(value) => setFilters((prev) =>
({ ...prev, severity: value }))}
                  >
                    <SelectTrigger>
                      <SelectValue placeholder="Severity" />
                    </SelectTrigger>
                    <SelectContent>
                                        <SelectItem   value="all">All
Severity</SelectItem>
                                                              <SelectItem
value="critical">Critical</SelectItem>
                      <SelectItem value="high">High</SelectItem>
                     <SelectItem value="medium">Medium</SelectItem>
                      <SelectItem value="low">Low</SelectItem>
                    </SelectContent>
                  </Select>
                  <Button
                    variant="outline"
                   onClick={() => setFilters({ search: "", building:
"all", status: "all", severity: "all" })}
                  >
                    Clear Filters
                  </Button>
                </div>
              </CardContent>
            </Card>
```

```
            {/* Reports Table */}
            <Card>
              <CardHeader>
                <CardTitle>Reports Management</CardTitle>
                <CardDescription>
                              Showing  {filteredReports.length}  of
{reports.length} reports
                </CardDescription>
              </CardHeader>
              <CardContent>
                <div className="overflowxauto">
                  <Table>
                    <TableHeader>
                      <TableRow>
                        <TableHead>ID</TableHead>
                        <TableHead>Reporter</TableHead>
                        <TableHead>Location</TableHead>
                        <TableHead>Issue Type</TableHead>
                        <TableHead>Severity</TableHead>
                        <TableHead>Status</TableHead>
                        <TableHead>Reported</TableHead>
                        <TableHead>Actions</TableHead>
                      </TableRow>
                    </TableHeader>
                    <TableBody>
                      {filteredReports.map((report) => (
                        <TableRow key={report.id}>
                                    <TableCell  className="fontmono
textsm">{report.id}</TableCell>
                          <TableCell>
                            <div>
                                                                <div
className="fontmedium">{report.name}</div>
                              {report.email && <div className="textsm
textgray500">{report.email}</div>}
                            </div>
                          </TableCell>
                          <TableCell>
                            <div>
                                                                <div
className="fontmedium">{report.building}</div>
                                                <div  className="textsm
textgray500">{report.room}</div>
                            </div>
                          </TableCell>
                          <TableCell>{report.issueType}</TableCell>
                          <TableCell>
                                                                <Badge
className={getSeverityColor(report.severity)}>
                           {report.severity.charAt(0).toUpperCase()
+ report.severity.slice(1)}
```

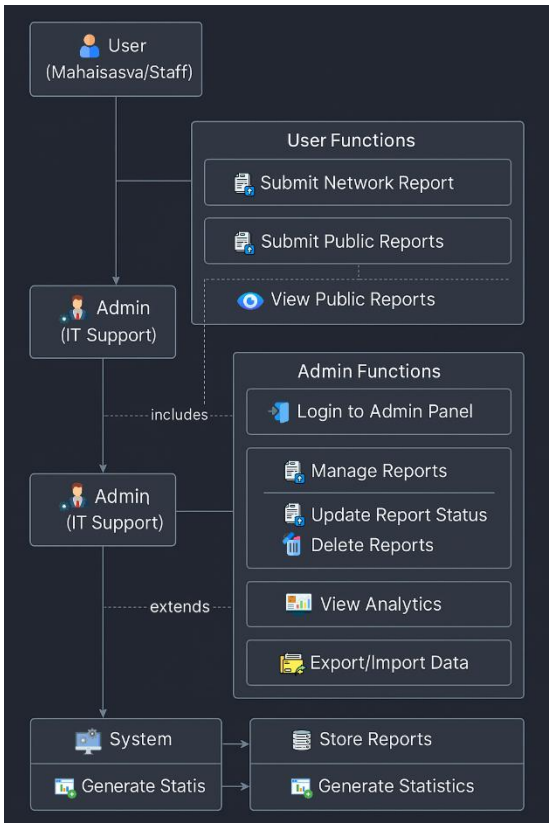
```
                              </Badge>
                            </TableCell>
                            <TableCell>
                                                                                <Badge
className={getStatusColor(report.status)}>
                                        {report.status.replace("_", " ").charAt(0).toUpperCase() +
                                         report.status.replace("_", " ").slice(1)}
                              </Badge>
                            </TableCell>
                                                                                <TableCell
className="textsm">{report.timestamp}</TableCell>
                            <TableCell>
                           <div className="flex itemscenter spacex2">
                                   <Select onValueChange={(value) => updateReportStatus(report.id, value)}>
                                 <SelectTrigger className="w32">
                                   <SelectValue placeholder="Update" />
                                 </SelectTrigger>
                                 <SelectContent>
                                                                                <SelectItem
value="new">New</SelectItem>
                                                                                <SelectItem
value="verified">Verified</SelectItem>
                                                                                <SelectItem
value="investigating">Investigating</SelectItem>
                                                                                <SelectItem
value="resolved">Resolved</SelectItem>
                                                                                <SelectItem
value="false_report">False Report</SelectItem>
                                 </SelectContent>
                               </Select>

                               <AlertDialog>
                                 <AlertDialogTrigger asChild>
                                 <Button variant="outline" size="sm" className="textred600 hover:textred700">
                                     <Trash2 className="w4 h4" />
                                   </Button>
                                 </AlertDialogTrigger>
                                 <AlertDialogContent>
                                   <AlertDialogHeader>
                                   <AlertDialogTitle className="flex itemscenter spacex2">
                                       <AlertCircle className="w5 h5 textred600" />
                                       <span>Delete Report</span>
                                     </AlertDialogTitle>
                                     <AlertDialogDescription>
                                     Are you sure you want to delete
```

```
report {report.id}? This action cannot be undone
                                        and will permanently remove
the report from the system.
                                    </AlertDialogDescription>
                                  </AlertDialogHeader>
                                  <AlertDialogFooter>
                                     <AlertDialogCancel>Cancel</Ale
rtDialogCancel>
                                    <AlertDialogAction
                                                   onClick={()  =>
deleteReport(report.id)}
                                               className="bgred600
hover:bgred700"
                                    >
                                      Delete Report
                                    </AlertDialogAction>
                                  </AlertDialogFooter>
                                </AlertDialogContent>
                              </AlertDialog>
                            </div>
                          </TableCell>
                        </TableRow>
                      ))}
                    </TableBody>
                  </Table>
                </div>
              </CardContent>
            </Card>
          </TabsContent>

          <TabsContent value="data" className="spacey6">
            <DataManagement />
          </TabsContent>
        </Tabs>
      </main>
    </div>
  )
}

export default function AdminDashboard() {
  return (
    <ProtectedRoute requireAdmin={true}>
      <AdminDashboardContent />
    </ProtectedRoute>
  )
}
```

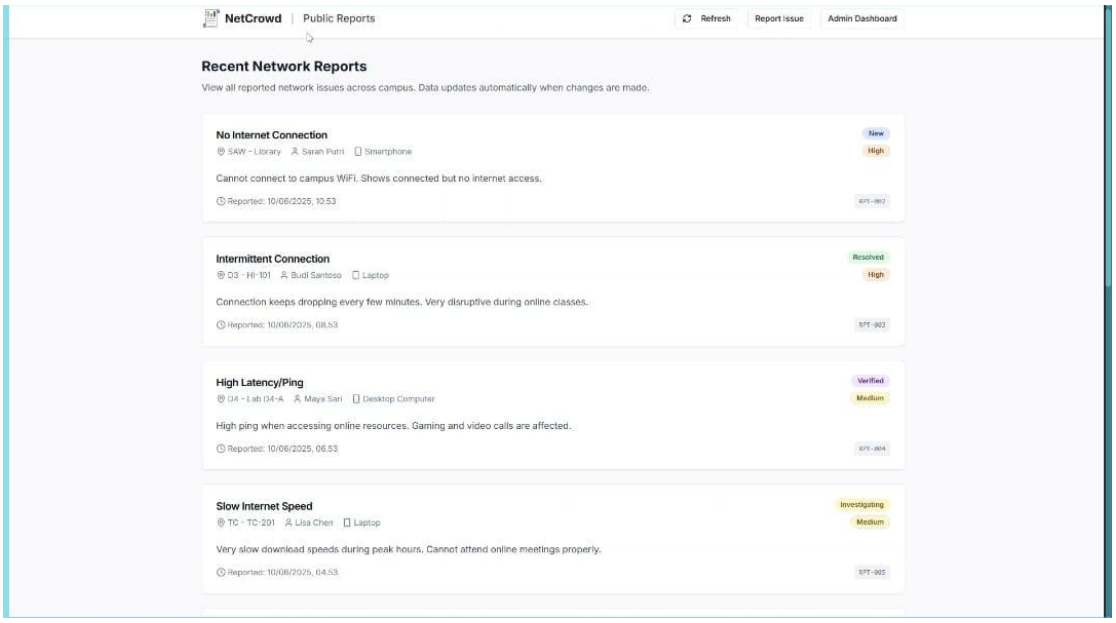
Program ini mendefinisikan halaman AdminDashboard yang merupakan dasbor komprehensif dan terlindungi yang hanya bisa diakses oleh administrator. Halaman ini berfungsi sebagai pusat kendali untuk sistem pelaporan jaringan "NetCrowd". Antarmukanya

menggunakan sistem navigasi berbasis *tab* yang memungkinkan admin beralih antara beberapa panel utama: "Overview" yang menyajikan statistik kunci, grafik, dan daftar gedung prioritas dengan masalah terbanyak; "Campus Map" yang mengintegrasikan komponen peta interaktif untuk pemantauan dan konfigurasi visual; "Manage Reports" yang menampilkan tabel laporan secara mendetail dengan fitur pencarian, filter, dan kemampuan untuk mengubah status atau menghapus laporan; serta "Data Management" untuk operasi data yang lebih luas. Komponen ini mengambil data secara *real-time* menggunakan *custom hooks* dari database, memberikan informasi terkini kepada admin, dan dibangun dengan serangkaian komponen UI modern dari shadcn/ui untuk memastikan pengalaman pengguna yang konsisten dan fungsional.

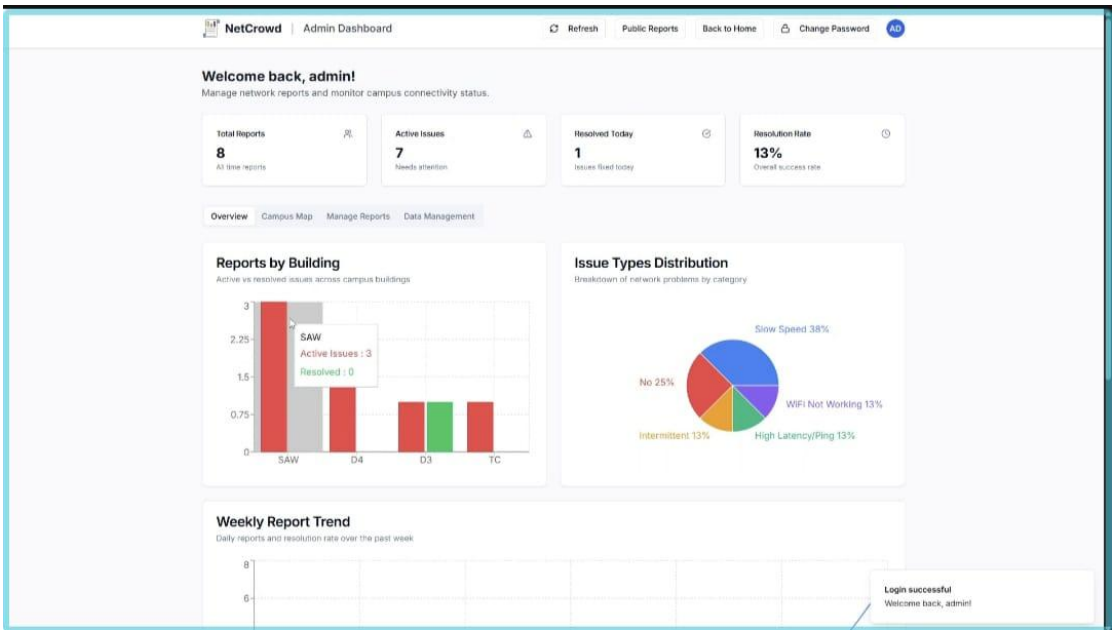
VI. Use Case Diagram



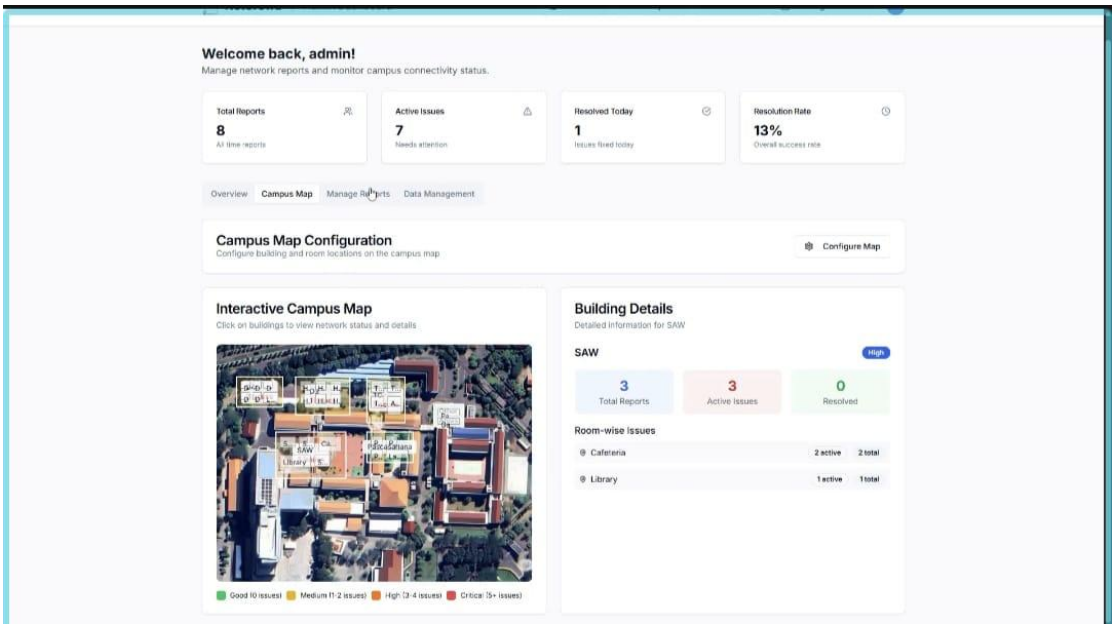
VII. Hasil Pengujian



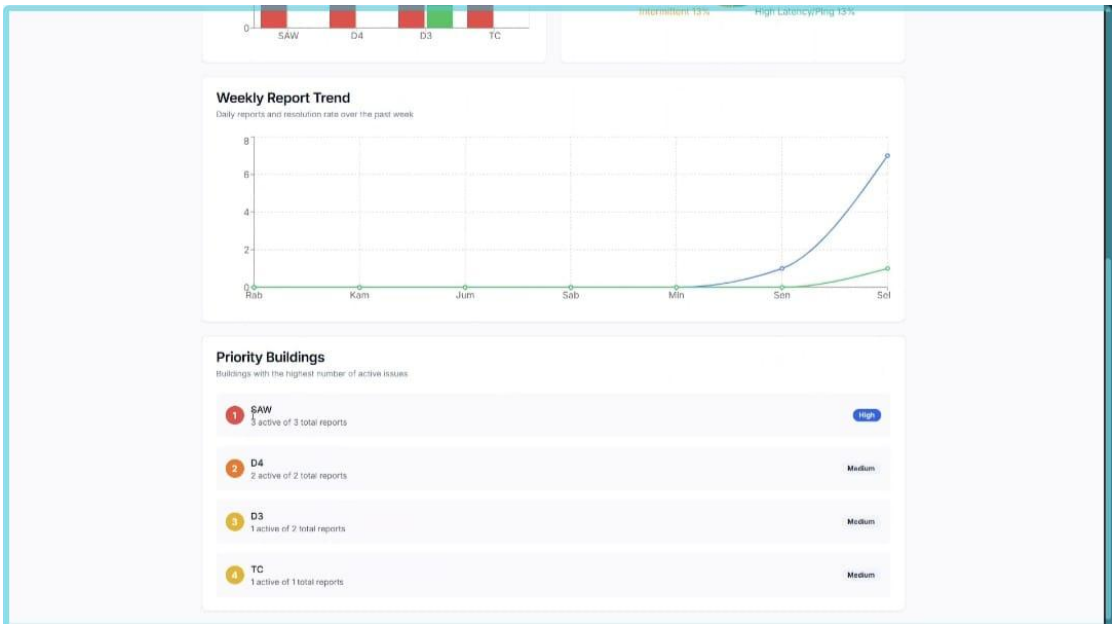
Pengguna dapat melihat report yang telah dikirim dan status reportnya



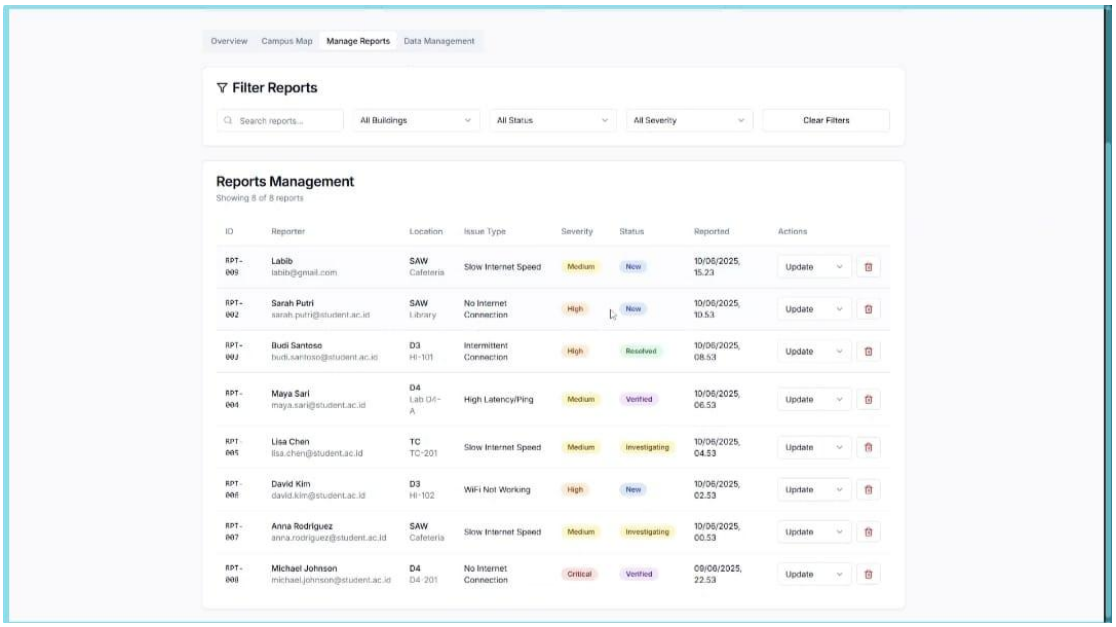
Admin bisa melihat hasil report yang divisualisasikan pada reports by building dan issues type distribution



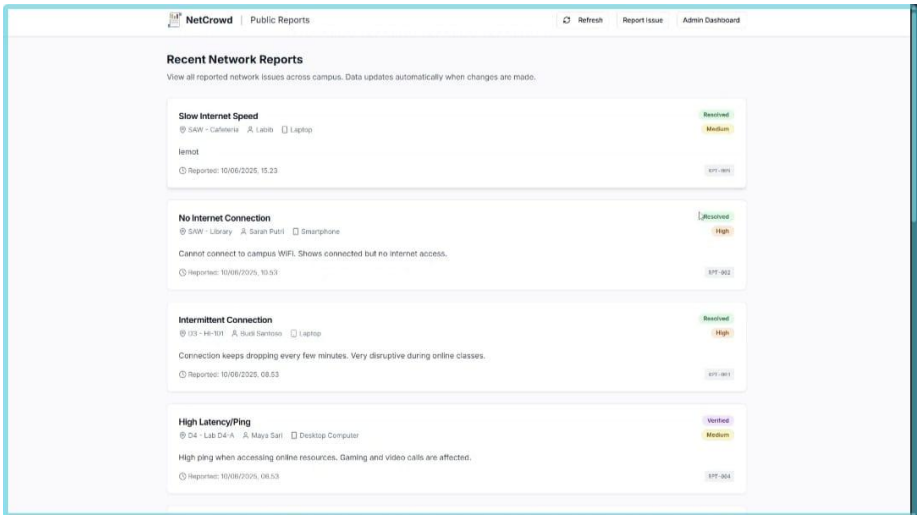
Admin dapat melihat peta interaktif yang bisa menujukkan informasi detail per gedungnya

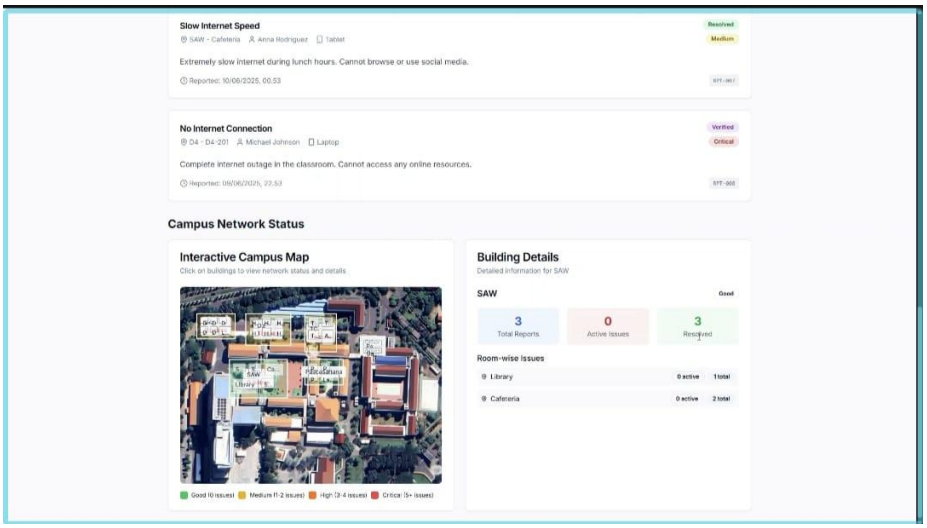


Admin bisa mengetahui Gedung yang harus diprioritaskan untuk segera diperbaiki



Admin bisa mengubah status report pada halaman reports management, jika status dirubah ke resolved maka dashboard status lainnya akan mengikuti





VIII. Kesimpulan

NetCrowd mampu menjadi pilihan dalam aplikasi pelaporan masalah jaringan kampus yang memudahkan pengguna untuk melaporkan gangguan koneksi secara rinci, mulai dari lokasi, jenis masalah, perangkat yang digunakan, hingga tingkat keparahan, sehingga membantu tim teknis dalam menangani isu jaringan secara cepat dan efisien.

IX. Lampiran

Link video: https://youtu.be/T47Y44KGzOw
Link repository Github: https://github.com/FoldExz/netcrowd
Link presentasi:
https://drive.google.com/file/d/1gcLrD6vs9XLGCBQYxOzHYvcir24yZF7P/view?usp=sharing